

SmartMeter Platform documentation

- [Kafka authentication using SASL/SCRAM](#)
 - [Introduction](#)
 - [Prerequisites](#)
 - [Kafka cluster configuration](#)
 - [Kafka topics dedicated to suppliers](#)
 - [Examples of Kafka Clients configuration](#)
- [Data reading](#)
 - [Schema definition](#)
 - [Converting AVRO to DataFrame](#)

Kafka authentication using SASL/SCRAM

This section of documentation provides detailed instructions on how to configure and authenticate to a Kafka cluster using SASL/SCRAM with Java and Python clients. Each client will have access to two dedicated topics and will use a specific group to consume data.

Introduction

SASL/SCRAM (Simple Authentication and Security Layer/ Salted Challenge Response Authentication Mechanism) is a mechanism for authenticating clients in Kafka. This guide will walk you through the steps required to set up SASL/SCRAM authentication for Kafka with Java and Python clients. Each client will be configured to access **two dedicated topics** and consume data using a **specific consumer group**.

Prerequisites

Before you begin, ensure you have the following:

- Kafka client tools (e.g., Kafka CLI, Java client, Python client)
- Properly configured client.properties files

Kafka cluster configuration

- Each cluster has **four brokers** :
 - **PROD**:
 - b1-smdata.reteleelectrice.ro:9001
 - b2-smdata.reteleelectrice.ro:9002
 - b3-smdata.reteleelectrice.ro:9003
 - b4-smdata.reteleelectrice.ro:9004
 - **PRE-PROD**:
 - b1-r-smdata.reteleelectrice.ro:9001
 - b2-r-smdata.reteleelectrice.ro:9002
 - b3-r-smdata.reteleelectrice.ro:9003
 - b4-r-smdata.reteleelectrice.ro:9004
- The Kafka cluster can be accessed by only using this **bootstrap URL**, afterwards the Kafka client will request the metadata from brokers.
 - **PROD**:
 - **smdata.reteleelectrice.ro:9000**
 - **PRE-PROD**:
 - **r-smdata.reteleelectrice.ro:9000**
- The certificate used for each DNS (bootstrap URL and brokers) is signed with Digicert Public CA and should be already trusted by many libraries.

Kafka topics dedicated to suppliers

Each supplier will have two dedicated topics, one for **LPO (Load Profile Order - load curves)** and another for **DCS (Daily Closures - daily readings)**.

The naming convention of topics contains the **Tax Code (CUI - the company's unique identification code)** and **type (LPO/DCS), all lowercase**. To consume data from the topics, the supplier will use its own dedicated consumer group (**group_<CUI>**), user (**user_<CUI>**) and password. **The credentials needed to access the cluster will be provided by Kafka administrators to the supplier.**

Example of **topics** and **credentials** for supplier with CUI RO1234567:

- **Topic names**: ro1234567_dcs, ro1234567_lpo
- **Consumer group**: group_ro1234567
- **User**: user_ro1234567, Password: xxxxxxxxxxxx - **password can contain single or double quotes, pay attention to escape it properly**

Examples of Kafka Clients configuration

- Kafka CLI

client.properties configuration file

```
# Kafka bootstrap URL address
bootstrap.servers=smdata.reteleelectrice.ro:9000

# Security protocol
security.protocol=SASL_SSL

# SASL mechanism
sasl.mechanism=SCRAM-SHA-512

# JAAS configuration for SASL/SCRAM
sasl.jaas.config=org.apache.kafka.common.security.scram.ScramLoginModule required \
  username="user_ro1234567" \
  password="xxxxxxxxxxxx";
```

Kafka command to consume topic data

```
./bin/kafka-console-consumer.sh --bootstrap-server smdata.reteleelectrice.ro:9000 --consumer.config client.properties --topic ro1234567_dcs --group group_ro1234567 --from-beginning
```

- Python

Library installation

```
pip install confluent-kafka
```

Python consumer

```
from confluent_kafka import Producer, Consumer, KafkaError

# Common configuration
bootstrap_servers = 'smdata.reteleelectrice.ro:9000'
sasl_mechanism = 'SCRAM-SHA-512'
sasl_plain_username = 'user_ro1234567'
sasl_plain_password = 'xxxxxxxxxxxxx'
security_protocol = 'SASL_SSL'

# Consumer configuration
consumer_config = {
    'bootstrap.servers': bootstrap_servers,
    'security.protocol': security_protocol,
    'sasl.mechanism': sasl_mechanism,
    'sasl.username': sasl_plain_username,
    'sasl.password': sasl_plain_password,
    'group.id': 'group_ro1234567',
    'auto.offset.reset': 'earliest'
}
consumer = Consumer(consumer_config)

# Subscribe to two topics and consume messages
consumer.subscribe(['ro1234567_dcs', 'ro1234567_lpo'])

while True:
    msg = consumer.poll(1.0)
    if msg is None:
        continue
    if msg.error():
        if msg.error().code() == KafkaError._PARTITION_EOF:
            continue
        else:
            print(msg.error())
            break
    print(f"Received message from {msg.topic():} {msg.value().decode('utf-8')}")

consumer.close()
```

- PySpark

PySpark consumer

```
from pyspark.sql import SparkSession
from pyspark import SparkContext, SparkConf

spark_conf = SparkConf()
context = SparkContext(conf=spark_conf)
spark = SparkSession(context).builder.appName(session_name).getOrCreate()
group = f"group_{kafka_credentials['topic']}"
df = (
    spark.read.format("kafka")
    .option("kafka.bootstrap.servers", kafka_credentials["bootstrap_server"])
    .option("subscribe", topic)
    .option("kafka.security.protocol", kafka_credentials["security_protocol"])
    .option("kafka.sasl.mechanism", kafka_credentials["sasl_mechanism"])
    .option("kafka.group.id", group)
    .option(
        "kafka.sasl.jaas.config",
        f'org.apache.kafka.common.security.scram.ScramLoginModule required username="{kafka_credentials["sasl_username"]}" password="{kafka_credentials["sasl_password"]}";',
    )
    .option("startingOffsets", "earliest")
    .option("endingOffsets", "latest")
    .load()
)
```

- Java

Java Consumer

```
import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.common.serialization.StringDeserializer;
import org.apache.kafka.common.serialization.StringSerializer;

import java.time.Duration;
import java.time.temporal.ChronoUnit;
import java.util.Arrays;
import java.util.Properties;

public class KafkaClientExample {
    public static void main(String[] args) {
        Properties props = new Properties();

        // Common properties
        props.put("bootstrap.servers", "smdata.reteleelectric.ro:9000");
        props.put("security.protocol", "SASL_SSL");
        props.put("sasl.mechanism", "SCRAM-SHA-512");
        props.put("sasl.jaas.config", "org.apache.kafka.common.security.scram.ScramLoginModule required
username=\"user_ro1234567\" password=\"xxxxxxxxxxxx\";");

        // Consumer specific properties
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
        props.put(ConsumerConfig.GROUP_ID_CONFIG, "group_ro1234567");
        KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props);

        // Subscribe to two topics
        consumer.subscribe(Arrays.asList("ro1234567_lpo", "ro1234567_dcs"));

        while (true) {
            ConsumerRecords<String, String> records = consumer.poll(Duration.of(100, ChronoUnit.MILLIS));
            for (ConsumerRecord<String, String> record : records) {
                System.out.printf("Consumed message from topic %s: %s%n", record.topic(), record.value());
            }
        }
    }
}
```

Data reading

Each message from Kafka represents an Avro message. Besides the additional Kafka data provided, such as timestamp of message injection in Kafka, offset, and others, the data generated can be found in the following two keys:

1. **KEY** - contains the schema of the AVRO message.
2. **VALUE** - contains the values of the AVRO message.

Currently, two different XML formats are processed:

1. **LPO** - Load Profile Order (i.e. curbe de sarcina)
2. **DCS** - Daily Closures (i.e. citiri zilnice)

Schema definition

To read Avro data, and interpret it in an usable format, one must define a schema. This is possible in two methods, static or dynamic.

Static schema - can be defined in files, variables, etc. Once this schema has been loaded in the processing script, the next part can be started. This needs to be a **Stringified JSON**, meaning that it must be a schema in a VALID JSON format, kept as a string.

For the two topics (<topic>_lpo, <topic>_dcs), the current Avro schema has been defined as it follows:

Data format - AVRO schema for LPO topic

```
{
  "type": "record",
  "namespace": "rer_lpo",
  "name": "rer_lpo",
  "fields": [
    {
      "name": "readDate",
      "type": "string"
    },
    {
      "name": "timestamp",
      "type": "string"
    },
    {
      "name": "meterSerialNumber",
      "type": "string"
    },
    {
      "name": "supplyPointId",
      "type": "string"
    },
    {
      "name": "energyType",
      "type": "string"
    },
    {
      "name": "dataValidation",
      "type": "string"
    },
    {
      "name": "sampleFrequency",
      "type": "string"
    },
    {
      "name": "complete",
      "type": "string"
    },
    {
      "name": "meterLastSampleDate",
      "type": "string"
    },
    {
      "name": "sampleDate",
      "type": "string"
    },
    {
      "name": "sampleValues",
      "type": "string"
    },
    {
      "name": "cui_furnizor",
      "type": "string"
    }
  ]
}
```

Data format - AVRO schema for DCS topic

```
{
  "type": "record",
  "namespace": "rer_dcs",
  "name": "rer_dcs",
  "fields": [
    {
      "name": "readDate",
      "type": "string"
    },
    {
      "name": "timestamp",
      "type": "string"
    },
    {
      "name": "meterSerialNumber",
      "type": "string"
    },
    {
      "name": "supplyPointId",
      "type": "string"
    },
    {
      "name": "energyType",
      "type": "string"
    },
    {
      "name": "dataValidation",
      "type": "string"
    },
    {
      "name": "sampleDate",
      "type": "string"
    },
    {
      "name": "sampleValues",
      "type": "string"
    },
    {
      "name": "cui_furnizor",
      "type": "string"
    }
  ]
}
```

Dynamic schema - each message published to Kafka contain both the values, but also the appropriate schema. Given that at the moment schema is static, the following code could be used to obtain the schema.

Assuming **DataFrame** has been defined earlier, when data was read from Kafka, the following Python function will produce the schema:

```

from pyspark.sql.dataframe import DataFrame

def get_schema(df: DataFrame) -> str:
    """
    Function that takes the dataframe.
    Based on the column 'key' that contains the schema, it takes the value from one row
    then converts it to a stringified JSON, which is returned.

    params:
    -----
    df : spark.DataFrame
        the dataframe containing data from Kafka

    returns:
    -----
    schema : str
        stringied JSON, contains schema to interpret AVRO
    """
    first_row = df.limit(1).collect()
    schema_bytearray = first_row[0]["key"]
    schema = schema_bytearray.decode("utf-8")
    schema = schema.split('||')[0]
    return schema

```

The variable obtained **schema** is a **Stringified JSON**.

Please bare in mind the split! For internal purposes, schema has concatenated to it a number.

Converting AVRO to DataFrame

The last step is to load from AVRO into usable format. The following code converts each message from AVRO, using a schema, into a new column named **values**. This is a nested column.

Lastly, for ease of use, this column is flattened, such that each key-value pair from inside the nested column can be easily accessible. This last step is optional.

```

from pyspark.sql.avro.functions import from_avro

schema = get_schema(df)

df_new = df.select(from_avro(df.value, schema).alias("values"))

df_new = df_new.select("values.*") # OPTIONAL

```